



CLUSTER LIFECYCLE & PRODUCTION READINESS

Book 09 — the finale. Building and maintaining the cluster itself: kubeadm bootstrap, etcd backup & restore (where every object you've ever applied actually lives), the upgrade choreography, GKE · EKS · AKS compared, multi-cluster thinking, and the exam map that turns nine books into three certifications.

 **kubeadm**
bootstrap

 **etcd**
the book of record

 **Snapshot**
backup + restore

 **Upgrade**
the choreography

 **GKE·EKS·AKS**
the big three

 **Multi-cluster**
fleet thinking

 **Exam map**
CKA·CKAD·CKS

 **Readiness**
the checklist

PART I · BIRTH & MEMORY

- ☐ **CH 01** How a Cluster Is Born — kubeadm p.3
- ☐ **CH 02** etcd — The Book of Record p.4
- ☐ **CH 03** Backup & Restore — The Sacred Drill p.5

PART II · GROWING OLD GRACEFULLY

- ☐ **CH 04** Cluster Upgrades — The Choreography p.6

PART III · THE MANAGED QUESTION

- ☐ **CH 05** Managed vs Self-Hosted p.7
- ☐ **CH 06** GKE · EKS · AKS — The Big Three p.8
- ☐ **CH 07** Multi-Cluster & Fleet Thinking p.9

PART IV · THE FINISH LINE

- ☐ **CH 08** The Exam Map — CKA · CKAD · CKS p.10
- ☐ **CH 09** The Production Readiness Checklist p.11

PART V · LABS

- ☐ **LAB 1** etcd Snapshot & Restore Drill p.12
- ☐ **LAB 2** The Upgrade Rehearsal p.13

PART VI · PROVE IT

- ☐ **QUIZ** Self-Check Quiz p.14
- ☐ **KEY** Answer Key + Scoring p.15
- ☐ **GLOSS** Glossary p.16
- ☐ **CHEAT** One-Page Cheat Sheet p.17
- ☐ **FINALE** The Series Complete p.18



Assumes Books 01–08. Book 08's finale promised this book would visit "where Helm's release receipts and every object ever applied actually live" — that's **etcd**, and CH 02–03 hand you its backup drill (the single most-tested CKA skill). Book 04's drain and Book 01's quorum both return as upgrade prerequisites.



Eight books assumed a cluster existed. Now: who builds it? A cluster is just **machines running seven binaries** (Book 01's cast) with a shared CA and a wired network. **kubeadm** is the official tool that does this plumbing — *not* a product, just a disciplined bootstrap.

```
$ kubeadm init \
  --control-plane-endpoint="k8s-api.example.com:6443" \
  --pod-network-cidr=10.244.0.0/16
# output: root CA · certs · static-pod manifests · the join command
$ kubectl apply -f calico.yaml # kubeadm does NOT install a CNI – you do
$ kubeadm join k8s-api.example.com:6443 --token ... --discovery-token-ca-cert-hash ...
# run on each worker → kubelet starts → node registers → Ready
```

What kubeadm init actually does

- └ generates PKI: CA, certs for apiserver/etcd/scheduler...
- └ writes static-pod manifests → /etc/kubernetes/manifests
 - (Book 04 · CH 11: kubelet boots the control plane!)
- └ starts etcd + CP components as static pods
- └ writes kubeconfig → /etc/kubernetes/admin.conf
- └ prints the join command (token + CA hash = trust on first use)



The bootstrap paradox, revisited: kubeadm doesn't "run" the control plane — it drops **manifest files** on disk, and the kubelet (the only thing already running) spawns the API Server as static pods. Book 04 · CH 11 explained why they're undeletable; now you know who wrote them.

Tool	For	Vibe
kubeadm	vanilla clusters on your VMs/metal	the official reference — CKA's stage
kops / Cluster API	infrastructure-aware creation (cloud, IaC)	kubeadm under the hood + lifecycle
managed (GKE/EKS/AKS)	someone else runs the CP (CH 05)	you run workers only
k3s / minikube / kind	edge, dev, testing	condensed — what this series ran on



One sentence: `kubeadm init` is **ceremony, not magic** — generate a CA, write static-pod manifests, let the already-running kubelet conjure the control plane from files, then hand out signed **join tokens** so workers can trust and be trusted.

▶ The thing kubeadm just booted and you must protect — [CH 02: etcd](#)

↑ back to top



Book 01 called etcd "the memory." Sharper: it's the **single source of truth** — every Deployment spec, every Secret, every Helm release receipt (Book 08 · CH 01!), every ServiceMonitor ever applied. **Back it up or you don't have a cluster — you have a cluster-shaped amnesia waiting to happen.**

**etcd gone, no backup**

every object lost; nodes keep running Pods that nobody can change; rebuild = re-declare everything from Git (if you have it...)

**etcd slow**

every API call waits — the whole cluster feels it (Book 07's "API slow" playbook row)

**etcd healthy + backed up**

quorum serving (Book 01 · CH 08) · snapshots on a schedule · tested restores

**etcd holds Secrets in plaintext by default**

(Book 02 · CH 07) — encryption-at-rest + locked-down PKI are etcd hygiene, not options

**Quorum math rules here too:**

3 members tolerate 1 failure; a

2-member etcd is the worst number

(Book 01's table, life-or-death edition)

**State separation (Book 05, finale-level):**

etcd = cluster state (specs, wishes). PVs = app data.

Back up BOTH

— an etcd restore with empty PVs is a database cluster full of empty databases

```
# talking to etcd directly — mTLS, always (its own certs!):
$ ETCDCCTL_API=3 etcdctl \
  --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key \
  endpoint health
https://127.0.0.1:2379 is healthy
```



Two CKA-favorite gotchas: (1) `ETCDCCTL_API=3` — the v2 API is ancient; snapshots don't exist in v2. (2) **The certs are mandatory** — etcd refuses unauthenticated clients by design; forgetting `--cacert/--cert/--key` fails with a connection error that looks like "etcd is down" when etcd is fine and you're just anonymous.



One sentence: etcd is the **book of record for the entire cluster** — everything Books 01–08 ever declared lives in its key-space — and its care routine is fixed: **quorum membership (odd counts), encryption-at-rest, mTLS access, and snapshots you have actually restored at least once.**

▶▶ The drill itself — [CH 03: Backup & Restore](#)

↑ back to top



Two commands. Guaranteed exam points. Career-saving muscle memory. Take the shot:

```
# BACKUP — one snapshot file, consistent point-in-time:
$ ETCCTL_API=3 etcdctl \
  --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key \
  snapshot save /backup/etcd-snapshot-$(date +%F).db
Snapshot saved at /backup/etcd-snapshot-2026-01-15.db

# VERIFY — never trust an unverified backup:
$ ETCCTL_API=3 etcdctl --write-table=json \
  snapshot status /backup/etcd-snapshot-2026-01-15.db
HASH      TOTAL KEYS  TOTAL SIZE
4a2c...   1842       6.2 MB
```



Restore is not "put the file back": `snapshot restore` builds a **new etcd data directory** (new cluster identity, written to a fresh `--data-dir`). The dance: **stop the API Server** → **stop etcd** → **restore to a new data-dir** → **point etcd at it** → **restart everything**. On a kubeadm cluster that means moving the static-pod manifests out, restoring, and moving them back — the API Server must be **down during restore**, because it would keep writing to the old, broken reality.

The restore choreography (single-CP)

```
1 • mv /etc/kubernetes/manifests/*.yaml /tmp/ → CP static pods die
2 • snapshot restore --data-dir=/var/lib/etcd-restored /backup/x.db
3 • edit etcd static-pod yaml → --data-dir=/var/lib/etcd-restored
4 • mv manifests back → etcd boots from the snapshot
5 • API Server returns → cluster = the snapshot's moment
multi-CP: restore ALL members from the SAME snapshot
(quorum must agree on one history — mixing timelines = split brain)
```

Backup reality check

The honest answer

Does a snapshot include PV data?	No — etcd stores specs, not bytes. Database data needs PV-level backups (Book 05 · CH 06 + volume snapshots)
Snapshot replaces GitOps?	No — they're a pair. Book 08: Git rebuilds declared state; snapshot recovers <i>exact</i> state including runtime-only fields (status, tokens, Helm receipts)
How often?	Scheduled (cron + etcdctl) and before every risky operation (CH 04's upgrades!)



One sentence: `snapshot save` + `snapshot status` take minutes and save careers — but restore is a **choreography** (quiesce → restore to a new data-dir → boot), and a backup you've never restored is a rumor, not a backup. Lab 1 runs the whole drill.

🔗 Keeping the cluster itself current — [CH 04: Upgrades](#)

↑ back to top



Kubernetes ships three releases a year; running a stale cluster means unsupported APIs and a painful jump eventually. The upgrade is a **fixed dance** — learn it once, perform it forever.



- 0 **Snapshot etcd** (CH 03) — the undo for the entire dance
- 1 **Control Plane:** upgrade the `kubeadm` binary → `kubeadm upgrade plan` → `kubeadm upgrade apply vX.Y` → upgrade `kubelet & kubectl` → restart kubelet
- 2 **Drain the CP node** (Book 04 · CH 10) → upgrade → **uncordon**
- 3 **Each worker, one at a time:** drain → `kubeadm upgrade node` → upgrade kubelet → uncordon → verify before the next

Rule	Why
One minor at a time (1.28 → 1.29, never → 1.30)	skew guarantees only hold between adjacent versions
API Server first, kubelets after	skew policy: kubelet may lag the API Server (up to 3 minors) — never lead it
Control plane before workers	new kubelets must speak to a new-or-equal API Server
Read the "Upgrade notes"	deprecated APIs get removed — your Book 08 CRDs and Helm charts may need version bumps <i>before</i> the cluster jumps

```
$ kubeadm upgrade plan
Components that must be upgraded: etcd · kubelet x3
You can now apply: v1.29.x
$ kubectl get nodes -o wide # watch versions march node-by-node:
# CP 1.29 while workers still 1.28 – HEALTHY SKEW (expected!)
```



Workloads ride through untouched: the upgrade moves the *platform*. Your Pods keep running — drain just asks them to walk across the street (Book 04). Only two things interrupt workloads: API removals (fix manifests first) and drain windows (spread workers, one at a time).



On managed clusters (CH 05–06): the vendor performs the CP upgrade — **you** still own node-pool upgrades and API-deprecation readiness. "Managed" moves the choreography, it doesn't delete it.



One sentence: upgrades are a **fixed choreography** — **snapshot, CP first (kubeadm apply), then workers one-by-one (drain → upgrade node → uncordon), one minor at a time, API Server never behind kubelets** — and the skill is boring repetition, not improvisation. Lab 2 rehearses it.



...unless you'd rather never run that dance — [CH 05: Managed vs Self-Hosted](#)

↑ back to top



The industry's default answer to CH 04's workload: **buy the control plane**. But "managed" is a spectrum — knowing exactly *which responsibilities transfer* is the interview-grade question.

Responsibility	Self-hosted (kubeadm)	Managed (GKE/EKS/AKS)
etcd — backup, quorum, encryption	you (CH 02–03)	vendor — invisible, SLA-backed
API Server / CP upgrade	you (CH 04's dance)	vendor performs CP; you upgrade node pools
CP availability (multi-master, LB)	you (Book 01 · CH 08's 3-node math)	vendor — regionally replicated
Cert rotation, security patching (CP)	you	vendor
Nodes, kubelets, node pools	you	you — still your drain/upgrade cycle
CNI, CSI, admission policy, workloads	YOU — Books 01–08, unchanged	



The one-line division: managed = "we run the brain" (CP + etcd). **The body is still yours** — nodes, networking policy, RBAC, workloads, observability. Everything from Books 01–08 except CH 02–03's etcd drills transfers over; and even there, **you own the data-residency and RBAC-around-etcd questions** in regulated environments.

✓ GO MANAGED WHEN...

your team is small; Kubernetes isn't your product; you want SLA-backed CP + integrated cloud IAM/storage (CH 06's payoff); ops headcount beats hardware budget.

✓ GO SELF-HOSTED WHEN...

edge/air-gapped/data-residency; cost at large scale (CP bill × 50 clusters); deep customization; *or* *Kubernetes IS your product* (you need kubeadm fluency — hence CKA).



One sentence: the managed question is really "**who owns the control plane's pager?**" — vendor takes etcd + CP availability + CP upgrades; **you keep every Books-01–08 responsibility on the node and workload side**, plus node-pool upgrades, which is why Book 04's drain skill survives every architecture choice.

▶▶ The three clouds side by side — [CH 06: GKE · EKS · AKS](#)

↑ back to top

Same Kubernetes, different clouds — the differences that actually bite live in three places: **networking model, IAM integration, node pools**.

	GKE (Google)	EKS (AWS)	AKS (Azure)
Default networking	VPC-native — Pod IPs are real VPC alias-IPs	AWS VPC CNI — Pod IPs drawn from the subnet (ENI limits!)	Azure CNI or kubenet — same IP-from-VPC tradeoff
IAM integration	Google IAM + Workload Identity	IRSA (IAM Roles for Service Accounts)	Entra ID + Workload Identity
Standout	Autopilot (Pod-level billing, no nodes to manage)	largest ecosystem; ALB controller pairs with Book 03's Ingress	smoothest Entra/AD enterprise fit

**Why the IAM piece matters to YOU:** it's Book 02's ServiceAccount story, cloud edition. A Pod that needs S3 access shouldn't hold static keys in a Secret (Book 02 · CH 07's risk) — it should **assume a cloud role via its ServiceAccount** (IRSA/Workload Identity). Workload identity = the SA token exchanged for short-lived cloud credentials. Same principle: *identity for workloads, scoped tight*.

- **Networking consequence:** VPC-native models mean **Pod IPs are routable cloud resources** — beautiful (no overlay) but **subnet-sized** : ENI/IP exhaustion is the classic EKS scaling failure; plan CIDRs like Book 03 taught you to plan Services
- **Node pools everywhere:** the Book 04 toolkit (taints, labels, drain) applies *per pool* — GPU pool tainted (Book 04 · CH 03), spot pool for batch (Book 06 · CH 09's CA loves spot)
- **Portability discipline:** everything in this series — vanilla YAML, Helm, Kustomize, GitOps — runs identically on all three. **Avoid vendor CRDs in your core manifests** ; keep cloud-specific bits in overlays (Book 08 · CH 04's structure earns its keep)

**One sentence:** the big three differ mainly in **how Pod IPs meet the cloud network, how ServiceAccounts meet cloud IAM, and how node pools are priced and shaped** — learn the mapping (CNI model · workload identity · pools) and you can move between all three; your Books-01–08 skills carry over unchanged.

When one cluster isn't the answer — [CH 07: Multi-Cluster](#)

↑ back to top



A single cluster is a **blast radius**. Bad deploys, bad upgrades (CH 04), regional outages — one control plane means one shared fate. Multi-cluster splits it. **Honest framing**: true "federation" (one API over many clusters) never won; today's practice is **independent clusters + shared tooling**.

★ Blast radius

prod-canary cluster takes the upgrade first (CH 04); a bad release never touches everything at once

🌐 Regions & residency

latency (route to nearest), data laws (EU data stays EU — a hard wall, not a preference)

🔗 Environments

the Book 08 overlay story, per-cluster instead of per-namespace

Challenge	Today's answer (concept level)
Deploy to many	GitOps fleet: ArgoCD ApplicationSets / Flux multi-cluster — one Git commit, N clusters reconcile (Book 08's pull model, fleet edition)
One DNS across clusters	global load balancing / multi-cluster Ingress (cloud LB + health checks steering between cluster edges — Book 03's LB at fleet level)
Trust between clusters	shared/mirrored CA or service-mesh identity — cross-cluster Service calls are an advanced, opt-in pattern , not default
Consistency	Git as the single spec (Book 08) + Book 09's snapshot discipline per cluster



The rule of thumb before going multi: don't — until a concrete driver exists (residency law, region-outage SLA, upgrade blast radius). Multi-cluster multiplies *everything*: $N \times$ upgrades, $N \times$ observability, $N \times$ etcd discipline. One well-run cluster beats five shaky ones; Book 07's dashboards and Book 08's GitOps are what make N survivable.



One sentence: multi-cluster today = **many independent clusters + GitOps fleet deployment + global traffic steering** — no magic single API — justified only by blast radius, residency, or reach, and affordable only because Books 07–08 made N clusters observable and declarative.

🔗 Cash in nine books — [CH 08: The Exam Map](#)

↑ back to top



Nine books map cleanly onto the three CNCF certifications. This page is your **study route converter**: each cert's domain list, the books that own it, and the lab skills to drill.

Cert	Focus	Series map	Drill these labs
CKAD developer	build & run workloads	B01 (Pods→Deployments) · B02 (config/secrets/probes...) · B05 (PVCs) · B06 (Jobs/HPA)	B01 L1–4 · B02 L1–4 · B05 L1 · B06 L2–3
CKA admin	operate clusters	B01 (architecture, quorum) · B04 (taints/drain) · B05 (PV lifecycle) · B03 (networking/CNI) · B09 (kubeadm, etcd, upgrades) · B07 triage	B09 L1–2 · B04 L4 · B05 L1–2 · B03 L3
CKS security (CKA prerequisite)	harden at runtime	B02 (RBAC, SecurityContext, PSS, Secret hygiene) · B03 (NetworkPolicy) · B07 (audit) · B08 (policy webhooks) + runtime security (falco, seccomp — beyond this series)	B02 L2–3 · B03 L3 · re-read B02 CH 12–13



All three are hands-on, terminal-based, ~2h exams — no multiple choice. The skills this series drilled as *runnable commands* are literally the exam tasks: `etcdctl snapshot save`, `kubectl auth can-i --as`, NetworkPolicy YAML, drain/uncordon, `kubectl taint`. Speed comes from the Lab muscle memory, not from reading.

Recommended sequence: this series = **CKAD + CKA body of knowledge in order** (Books 01–09); then add runtime-security reading on top for CKS. Book 01's "exam tip" callouts, every box, and each book's quiz were written against these blueprints — re-do the quizzes under time pressure as your final exam-prep pass.



One sentence: CKAD = **Books 01–02 + 05–06** (build), CKA adds **04 + 09 + 07** (operate), CKS = **02 + 03 + 08's webhooks** (harden) — three exams, one coherent series, and every lab you already ran was exam practice in disguise.

▶▶ One page to govern them all — [CH 09: The Production Checklist](#)

↑ back to top

The Production Readiness Checklist

Nine books, compressed into the questions an SRE asks before saying "yes, ship it." Every line cites its owning book — this is the whole series as a gate.

CLUSTER & ETCD (B01 · B09)

- ☑ 3+ CP nodes — quorum math sane (B01-CH08)
- ☑ etcd **scheduled AND** (B09-CH03)
snapshots **restored once**
- ☑ Secrets encrypted at rest (B02-CH07)
- ☑ Upgrade path rehearsed; versions ≤ 1 minor behind (B09-CH04)

IDENTITY & ACCESS (B02)

- ☑ No wildcard RBAC; least-privilege SAs per workload
- ☑ default SA cannot do anything (B02-CH11)
- ☑ PSS enforce on prod namespaces (B02-CH13)
- ☑ No raw Secrets in Git; external-secrets/SOPS

TRAFFIC & ISOLATION (B03)

- ☑ Ingress + TLS at the edge; no raw NodePort prod
- ☑ default-deny NetworkPolicy + named allows; DNS allowed first
- ☑ CNI actually enforces policies (verified, not assumed)

WORKLOADS & DATA (B04–06)

- ☑ Probes on everything (B07-CH02–04)
- ☑ requests/limits set → QoS deliberate (B02-CH14–15)
- ☑ Stateful data on PVCs, Retain policy; PV backup story (B05)
- ☑ PDBs before the first real drain (B04-CH10)

OBSERVABILITY (B07)

- ☑ Metrics + logs central; dashboards exist
- ☑ Alerts route to humans who sleep in shifts
- ☑ Triage reflex drilled: nodes → pods -A → events

DELIVERY (B08)

- ☑ Everything declarative & in Git; GitOps reconciling
- ☑ helm template / -k preview in CI before apply
- ☑ Rollback path proven (helm rollback / rollout undo)



One sentence: production readiness isn't a product you buy — it's **this checklist, all green**: survivable control plane (B01/09), least-privilege identity (B02), sealed edges and rooms (B03), healthy and persistent workloads (B04–06), truth-telling observability (B07), and a delivery system that reconciles (B08). **Every ☑ is a chapter you can now defend in an interview.**

 Run it with your hands — [Part V: Labs](#)

↑ back to top

**STEP 1 · Reach etcd inside minikube (docker driver)**

```
$ minikube ssh
$ docker exec -it etcd-minikube sh
# you're inside the etcd static-pod's container -
# its certs are mounted at /etc/kubernetes/pki/etcd/
```

STEP 2 · Take + verify the snapshot

```
$ ETCDCTL_API=3 etcdctl \
  --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key \
  snapshot save /backup/snap1.db
Snapshot saved at /backup/snap1.db
$ ETCDCTL_API=3 etcdctl --cacert=... --cert=... --key=... \
  snapshot status /backup/snap1.db --write-table
TOTAL KEYS  TOTAL SIZE
~1800        ~6 MB  # keys = every object you've ever applied!
```

00 PROVE IT'S REAL STATE

exit container; from host kubectl: \$ kubectl
create ns snapshot-proof \$ kubectl get ns |
grep proof snapshot-proof Active # this ns
exists NOW but not in # the snapshot taken a
moment ago — # a restore would erase it.
that's # "point-in-time," felt firsthand

⚠️ RESTORE — THE HONEST PART

full restore on minikube = rebuild # (single-
node CP, static pods). # the DRILL to know
cold: 1 · mv manifests out (CP dies) 2 · etcdctl
snapshot restore \ --data-dir=/var/lib/etcd-
new \ snap1.db 3 · point etcd pod at new
data-dir 4 · manifests back → boot # memorize
the ORDER; the exam # runs it on a 3-node CP
where it # works exactly like CH 03

Proved: mTLS access, snapshot+status, and point-in-time semantics. **Cleanup:** `kubectl delete ns snapshot-proof`.

[↑ back to top](#)



Honest scope: minikube won't perform a real kubeadm upgrade — but the **surrounding choreography** (inventory, skew reading, drain discipline, pre-flight snapshot) is 80% of the real dance and fully drillable here.

STEP 1 · Version inventory — the skew table

```
$ kubectl get nodes -o wide
NAME      STATUS  VERSION  OS-IMAGE
minikube  Ready   v1.2x.x   Ubuntu...
$ kubectl version --short 2>/dev/null || kubectl version
# read it like CH 04: is any kubelet NEWER than
# the API server? (that's the forbidden direction)
```

STEP 2 · Pre-flight snapshot (the step-0 of every upgrade)

```
$ # LAB 1's snapshot command, rerun — labeled for the upgrade:
# /backup/pre-upgrade-$(date +%F).db
# rule: no snapshot, no upgrade. ever.
```

STEP 3 · The node-march, rehearsed

```
$ kubectl cordon minikube && kubectl get nodes
STATUS
Ready,SchedulingDisabled # step 2 of the dance (B04·CH10)
$ kubectl uncordon minikube
$ kubectl get pods -A --no-headers | wc -l
# your workload census — after a REAL upgrade you'd
# re-run this and the Book 07 triage reflex to verify
# everything survived the march
```

Proved: skew reading, snapshot-before-change discipline, and the cordon→uncordon march. The real `kubeadm upgrade` apply is this rehearsal + one command on a cluster you own.

[↑ back to top](#)



Q1. `kubeadm init` starts the control plane by...

- A.** running apiserver as a systemd service **B.** writing static-pod manifests the already-running kubelet picks up **C.** installing Docker images manually **D.** calling the cloud API

Q2. What does an etcd snapshot contain?

- A.** All PV data **B.** Cluster state — every object's spec & metadata **C.** Container images **D.** Only RBAC rules

Q3. Before `snapshot restore`, you must...

- A.** delete all namespaces **B.** upgrade kubeadm first **C.** stop the API Server (and etcd), then restore to a new data-dir **D.** nothing — restore is hot

Q4. Upgrading a cluster 1.28 → 1.30 in one step is...

- A.** recommended **B.** fine for managed clusters **C.** forbidden — one minor version at a time **D.** required for kubeadm

Q5. Valid version skew during an upgrade?

- A.** kubelet newer than API Server **B.** kubelet older than API Server (never newer) **C.** kubelet equal always **D.** skew doesn't matter

Q6. On a managed cluster (EKS), who upgrades the node pools' kubelets?

- A.** the vendor, automatically always **B.** nobody — not needed **C.** you — via managed node-group upgrades (drain discipline still yours) **D.** the CNI

Q7. EKS/AKS/GKE's "workload identity" is the cloud edition of...

- A.** NetworkPolicy **B.** ServiceAccounts assuming scoped cloud roles — no static keys in Secrets **C.** Taints **D.** ResourceQuota

Q8. Today's multi-cluster practice is best described as...

- A.** one global kubectl API over all clusters **B.** independent clusters + GitOps fleet deployment + global traffic steering **C.** a single giant etcd **D.** deprecated kubefed everywhere

Q9. The mandatory etcdctl flags for any snapshot command are...

- A.** none — open by default **B.** `--endpoints` + `CA/cert/key` (mTLS) + `ETCDCTL_API=3` **C.** only a password **D.** `--insecure`

Q10. CKA's most heavily weighted hands-on area maps to which part of this series?

- A.** Helm templating **B.** cluster administration — etcd backup/restore, upgrades, networking, troubleshooting (B03/04/09) **C.** sidecar patterns **D.** GitOps UIs

[↑ back to top](#)



Q	Ans	Why — one line
1	B	The bootstrap paradox: kubelet + manifest files conjure the CP — CH 01
2	B	Specs & metadata only; PV bytes need their own backup — CH 02/03
3	C	Restore = quiesce, restore to NEW data-dir, boot — CH 03, Lab 1
4	C	Skew guarantees hold only between adjacent minors — CH 04
5	B	kubelet may lag, never lead, the API Server — CH 04
6	C	Vendor does CP; node pools stay your drain-cycle duty — CH 05/06
7	B	SA → cloud role, short-lived creds — Book 02's identity model, cloud edition — CH 06
8	B	Federation never won; fleets + steering won — CH 07
9	B	mTLS by design + v3 API — the anonymous-client trap — CH 02, Lab 1
10	B	CKA = operate: etcd, upgrades, network, triage — CH 08's map

**9–10**

Series mastered — exam-ready

**6–8**Re-read missed chapters'  boxes**<6**

Re-run Labs 1–2, then retake



Missed Q3? The restore choreography is *the* CKA etcd task — order matters more than flags. Missed Q6? "Managed" moving-but-not-deleting the choreography is the conceptual hinge of Part III.

[↑ back to top](#)



kubeadm — official bootstrap tool: generates PKI, writes static-pod manifests, prints join tokens.

Join token / CA hash — trust-on-first-use credentials letting a worker's kubelet register securely.

etcd — the cluster's key-value source of truth; quorum-based (odd members); holds every object incl. Helm receipts.

etcdctl — etcd client; v3 API required for snapshots; mTLS flags mandatory.

snapshot save / status / restore — consistent backup → verify → rebuild data-dir (new identity, API Server down during).

Version skew — supported mixing during upgrades; kubelet ≤ API Server, adjacent minors only.

kubeadm upgrade apply / node — CP-side / worker-side upgrade commands; one minor per hop.

Managed Kubernetes — vendor runs CP + etcd (GKE/EKS/AKS); you own nodes, CNI/CSI choices, workloads.

Node pool — a group of like nodes (per taints/labels/size) managed as a unit — the drain/upgrade unit in clouds.

Workload Identity / IRSA — ServiceAccount-assumed cloud roles; short-lived creds instead of static keys.

VPC-native / VPC CNI / Azure CNI — Pod IPs drawn from the cloud network (no overlay) — routable but subnet-bound.

Multi-cluster — many independent clusters; fleet GitOps (ApplicationSets) + global LB steering; no unified API.

Blast radius — how much one failure touches; the reason clusters (and upgrades) are split.

CKA / CKAD / CKS — CNCF hands-on certs: operate / build / harden — mapped in CH 08.

[↑ back to top](#)



ETCD — THE SACRED COMMANDS

```
ETCDCTL_API=3 etcdctl \
  --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key \
  snapshot save /backup/snap.db
$ ... snapshot status snap.db --write-table
restore: stop APISERVER → snapshot restore
  --data-dir=/var/lib/etcd-new → repoint
  etcd → boot. multi-CP: same snapshot
  on ALL members.
```

UPGRADE CHOREOGRAPHY

```
0 • SNAPSHOT (no snapshot, no upgrade)
1 • CP: kubeadm upgrade plan
  kubeadm upgrade apply v1.X
  upgrade kubelet+kubect1, restart
2 • drain CP → uncordon
3 • per worker (one at a time):
  drain → kubeadm upgrade node
  → kubelet → uncordon → verify
ONE MINOR per hop • kubelet never
newer than apiserver • read API-removal
notes BEFORE jumping
```

THE BIG THREE

	Network	Workload ID
GKE	VPC-native	Workload Identity
EKS	AWS VPC CNI	IRSA
AKS	Azure CNI/kubenet	Entra + WI

```
managed = vendor runs CP+etcd;
YOU run node pools, CNI policy, workloads
– Books 01–08 skills transfer 100%
```

EXAM MAP

```
CKAD: B01-02-05-06 (build)
CKA:  + B03-04-07-09 (operate –
      etcd/upgrades/network/triage)
CKS:  B02-03-08-webhooks
      + runtime security (harden)
all hands-on: the LABS are the prep
```

FACTS WORTH A JOB OFFER

kubeadm = PKI + static-pod manifests + join tokens; the kubelet boots the control plane from files

etcd snapshots = cluster state only (never PV data); restore is cold — API Server down, new data-dir, same snapshot on all members

Upgrades: snapshot → CP first → workers one-by-one; one minor per hop; kubelet may lag, never lead

Managed = vendor owns CP pager; you keep node pools, drain discipline, CNI policy, and every Books-01–08 duty

Cloud trio differs by CNI model + workload identity (SA→cloud role, no static keys) + node pools

Multi-cluster = independent clusters + GitOps fleet + global steering — justified by blast radius/residency only

[↑ back to top](#)



	THE NINE-BOOK ARC — one machine, taught layer by layer:	
— 01 Architecture	brain + muscle · reconcile loop · namespaces	
— 02 Config & Access	configmaps · secrets · RBAC · QoS	
— 03 Networking	services · ingress+TLS · policies · DNS	
— 04 Scheduling	taints · affinity · priority · drain	
— 05 Storage	PV/PVC · classes · stateful data · CSI	
— 06 Workloads	daemonsets · jobs · canaries · autoscaling	
— 07 Observability	probes · prometheus · loki · triage	
— 08 Delivery	helm · kustomize · CRDs · operators · gitops	
— 09 Lifecycle	kubeadm · etcd · upgrades · clouds · exams	

You started with "what is a cluster?" — a brain and muscle. You end able to build the brain, back it up, upgrade it, place workloads on the muscle, connect them, observe them, and let Git keep the whole story true. **That is the complete operator's arc.**

Where to go from here

Certify: CH 08's map — sit CKAD, then CKA; add runtime security reading for CKS

Practice: rebuild a multi-node cluster with kubeadm (3 VMs) — CH 01's ceremony, felt for real; then break it and restore from Lab 1's snapshot

Ship: run one real project the full Book-08 way — Helm or Kustomize + GitOps + Book-07 observability, on GKE/EKS/AKS (CH 06's notes apply)

Go deeper: service meshes, eBPF/Cilium, multi-tenancy, platform engineering — every one of them stands on these nine layers



The last word, in the series' oldest joke's spirit: Kubernetes is just a reconciliation loop that refuses to give up on your wishes. You now know every kind of wish it accepts, every gate they pass, and every way it keeps its promises. **Declare wisely.**

— End of Book 09 · End of Series · —